

Лекция 1

LLM через API и промптинг

как разговаривать с моделью из кода, платить за это осознанно и получать ответ
в нужной форме

- ❶ **Провайдеры и API**: кто есть кто, как устроен запрос и ответ
- ❷ **Токены и деньги**: за что платим и как считать заранее
- ❸ **Промптинг**: системный промпт, примеры, рассуждение
- ❹ **Structured output**: ответ как данные, а не как текст

Зачем это агенту

Агент это цикл вокруг модели. Прежде чем строить цикл, надо уверенно делать один вызов: понимать, что ушло в модель, что вернулось и сколько это стоило.

Чат в браузере

- Один человек, один диалог
- Настройки спрятаны
- Ответ читает человек
- Нельзя встроить в продукт

Модель через API

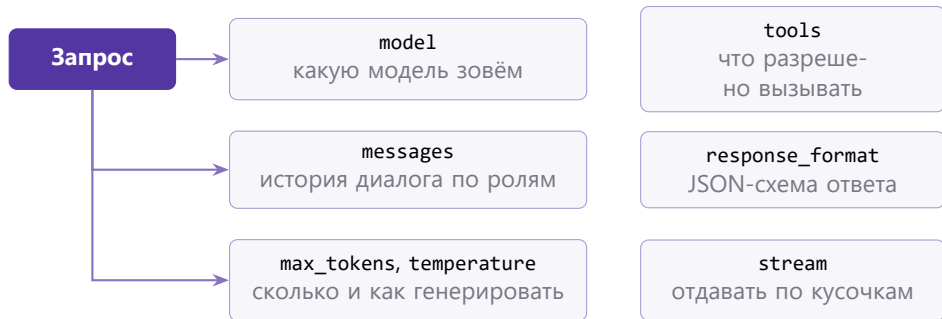
- Тысячи вызовов в час из кода
- Полный контроль: температура, длина, формат
- Ответ читает **программа**
- Можно дать модели инструменты

Главное отличие

В чате модель говорит с вами. Через API модель становится **функцией**: текст на входе, текст или JSON на выходе, и эту функцию можно вызывать в цикле.

Кто	Что даёт	Зачем брать
Anthropic	семейство Claude	сильный tool use, длинный контекст
OpenAI	семейство GPT	самый распространённый формат API
Google	Gemini	дешёвый Flash, контекст до миллиона
Open-weight	Llama, Qwen, Mistral, DeepSeek	можно хостить самим, дёшево
OpenRouter	все модели выше через один ключ	смена модели одной строкой

- У каждого свой SDK, но почти все умеют формат OpenAI chat completions
- На курсе работаем через OpenRouter: один ключ, одна форма запроса, любая модель
- Прямой SDK провайдера нужен, когда используете его особые фишки: кэш, батчи, файлы



Всё это один HTTP POST с JSON-телом. SDK лишь прячет его за красивой функцией.

Роли: что на самом деле видит модель

читается первым

system

кто ты, правила, формат

user

первый вопрос

assistant

прошлый ответ модели

user

новый вопрос

генерация начинается отсюда

- Модель ничего не помнит между вызовами. **Память это ваш список messages**
- Каждый вызов получает всю историю и платит за неё заново
- system это инструкция с приоритетом, но не магия: для модели это просто текст сверху
- assistant это её прошлые ответы. Сюда же попадут вызовы инструментов

Первый вызов: без SDK, просто HTTP

```
import requests, os

r = requests.post(
    "https://openrouter.ai/api/v1/chat/completions",
    headers={"Authorization": f"Bearer {os.environ['OPENROUTER_API_KEY']}"},
    json={
        "model": "anthropic/claude-haiku-4.5",
        "messages": [
            {"role": "system", "content": "Отвечай одним предложением."},
            {"role": "user", "content": "Что такое tool calling?"},
        ],
        "max_tokens": 200,
        "temperature": 0.2,
    },
    timeout=60,
)
data = r.json()
print(data["choices"][0]["message"]["content"])
print(data["usage"])
```

Двадцать строк, и это весь «искусственный интеллект» с точки зрения кода. Всё остальное на курсе будет надстройкой над этим вызовом.

```
{
  "choices": [{
    "message": {"role": "assistant", "content": "Tool calling это ..."},
    "finish_reason": "stop"
  }],
  "usage": {"prompt_tokens": 41, "completion_tokens": 27, "cost": 0.00018}
}
```

- content: сам текст. Может быть пустым, если модель решила вызвать инструмент
- finish_reason: stop значит закончила сама, **length** значит упёрлась в max_tokens и ответ обрезан
- usage: токены и деньги. **Читать всегда:** без этого не поймёте, куда ушёл бюджет

Токены: единица, в которой всё измеряется

- Токен это кусок текста: слово, часть слова, знак
- Английский: примерно 0,75 слова на токен
- **Русский дороже**: 1,5–2 токена на слово, кириллица режется мельче
- Контекстное окно измеряется в токенах: 200 тысяч, 1 миллион

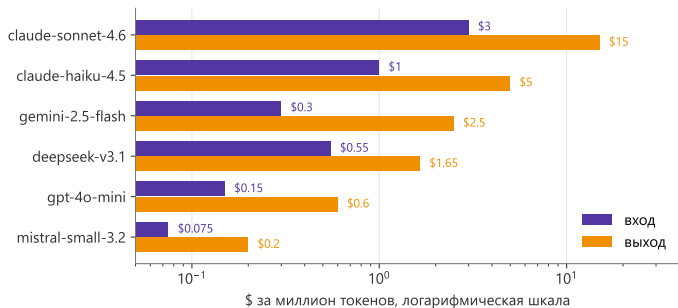
Токены на входе и на выходе считаются отдельно и стоят по-разному. Это следующий слайд.

Прикидка на глаз

Страница текста А4 это около 500 слов.
По-английски примерно 700 токенов.
По-русски примерно **1000** токенов.

Точно считает `count_tokens` у провайдера или библиотека `tiktoken`.

СКОЛЬКО СТОИТ ВЫЗОВ



$$\text{цена} = \frac{\text{ВХОД} \cdot p_{in} + \text{ВЫХОД} \cdot p_{out}}{10^6}$$

- Выход в 3–5 раз дороже входа
- Разброс между моделями в **сто раз**
- Страница вопроса и абзац ответа: от 0,01 до 0,6 цента
- Один вызов дешёв. Тысяча задач по десять шагов уже нет

Цены OpenRouter, август 2026. Меняются, проверяйте перед расчётом бюджета.

- **temperature** 0–1: разброс вариантов. Ноль почти детерминирован, единица творчески
- **max_tokens**: жёсткий потолок ответа. Мало: обрезка. Много: ничего, платите за факт
- **top_p**: альтернатива температуре, вместе их не крутят
- **stop**: строки, на которых генерация останавливается

У reasoning-моделей часть параметров отключена или заменена уровнем усилия. Читайте документацию конкретной модели.

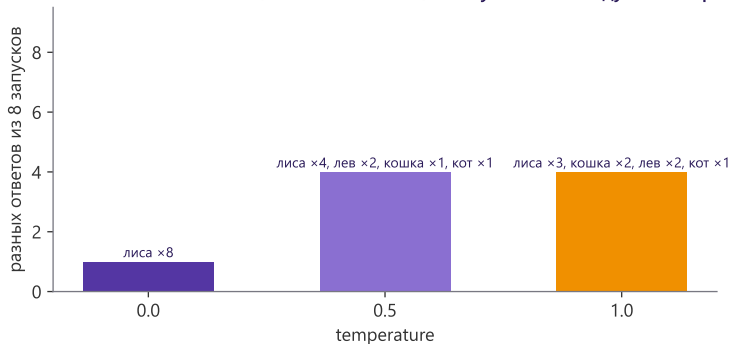
Для агентов и извлечения данных

temperature 0–0.3

max_tokens с запасом, но не бесконечный
никакого top_p

Творческий режим нужен для текстов, не для решений, какой инструмент вызвать.

«Назови любое животное», claude-haiku-4.5, 8 запусков на каждую температуру



- При нуле ответы почти не меняются между запусками: то, что нужно, когда результат читает программа
- При единице та же модель отвечает по-разному: хорошо для текстов, плохо для извлечения данных

Четыре части хорошего системного промпта:

- 1 **Роль**: кто модель и для кого работает
 - 2 **Задача**: что именно делать и чего не делать
 - 3 **Формат**: как выглядит ответ, длина, язык, структура
 - 4 **Границы**: что делать, если данных нет, если просят лишнее
- Промпт это код: версионировать, тестировать на фиксированном наборе, не правьте «на глаз»
 - Инструкции в стиле «ты гениальный эксперт мирового уровня» ничего не добавляют

Пример

Ты помощник поддержки интернет-магазина. Отвечай по-русски, не длиннее трёх предложений. Опирайся только на переданные документы. Если ответа в них нет, скажи об этом прямо и предложи связаться с оператором.

Few-shot: когда примеры лучше инструкций

- Формат сложно описать словами, но легко показать: 2–5 примеров вход/выход
- Примеры подаются как пары user/assistant в истории или блоком в системном промпте
- Порядок и разнообразие примеров важнее их числа
- Каждый пример стоит токенов на **каждом** вызове

Где few-shot выигрывает

Классификация с редкими метками
Извлечение полей в свой формат
Стиль текста, который не объяснить

Где проигрывает

Задачи с рассуждением: пример подсказывает форму, а не логику.
Лучше чёткая инструкция плюс structured output.

Chain-of-thought вчера

- Фраза «рассуждай пошагово» заметно поднимала точность
- Рассуждение шло в ответ и стоило выходных токенов
- Его надо было отрезать от финального ответа

Reasoning-модели сегодня

- Рассуждают до ответа сами, это отдельный режим
- Управляется уровнем усилия, а не фразой в промпте
- Рассуждение оплачивается, даже если не показано

Что это значит для агента

В цикле ReAct мы всё равно просим модель написать одну фразу «что делаю и зачем» перед вызовом инструмента. Не ради точности, а ради **читаемого трейса**: иначе непонятно, почему агент свернул не туда.

Проблема

- Модель отвечает текстом, программа ждёт поля
- Парсить регулярками: ломается на первом «конечно, вот JSON:»
- Просить «верни JSON» в промпте: помогает, но не гарантирует

Решение по нарастающей

- 1 **JSON mode**: провайдер гарантирует валидный JSON, но не поля
- 2 **Схема**: передаёте JSON Schema, модель обязана ей следовать
- 3 **Pydantic**: описываете класс, схема генерируется, ответ валидируется

Правило

Всё, что читает программа, а не человек, должно приходить по схеме. Tool calling, который мы разберём в третьей лекции, это тот же structured output, только вызов функции.

Structured output в коде

```
from pydantic import BaseModel, ValidationError

class Ticket(BaseModel):
    category: str          # "доставка" | "возврат" | "оплата" | "другое"
    urgency: int           # 1..5
    summary: str

schema = Ticket.model_json_schema()
body["response_format"] = {"type": "json_schema",
                           "json_schema": {"name": "ticket", "schema": schema}}

raw = call_model(body)          # текст ответа
try:
    ticket = Ticket.model_validate_json(raw)  # объект или исключение
except ValidationError as e:
    ticket = retry_with_error(body, raw, e)    # один повтор с текстом ошибки
```

Повтор с текстом ошибки валидации чинит большинство сбоев: модель видит, какое поле не то, и исправляет.

- **429**: превышен лимит запросов.
Подождать и повторить
- **500–503**: провайдер лёг. Повторить с растущей паузой
- **Таймаут**: длинные ответы идут минуту и больше. Ставьте таймаут явно
- **Обрезка**: `finish_reason = length`.
Поднять `max_tokens` или укоротить запрос

Минимальный набор

Повтор 2–3 раза с паузой 1, 2, 4 секунды

Таймаут 60–120 секунд

Логировать каждый вызов: модель, токены, цена, ошибка

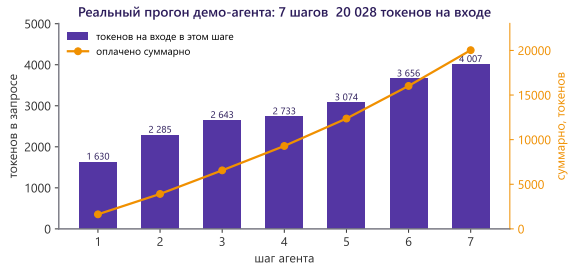
Запасная модель на случай, если основная недоступна

Streaming: ответ приходит по кусочкам. Нужен для чатов с людьми и для очень длинных ответов. Агенту между шагами обычно не нужен.

Контекстное окно: большое не значит бесплатное

- Окно в миллион токенов не повод класть туда всё
- Каждый вызов платит за весь контекст: длинная история дорожает линейно
- Качество падает: важное в середине длинного контекста теряется чаще, чем в начале и конце
- Модель не отличает данные от инструкций внутри контекста

Отсюда вся вторая неделя: память, суммаризация, RAG. Это способы **не тащить всё** в каждый вызов.



Контекст накапливается: каждый шаг дороже предыдущего, а платим за сумму.

Prompt caching: платить за префикс один раз

- Провайдер кэширует начало запроса: системный промпт, схемы инструментов, документы
- Повторный вызов с тем же префиксом платит за него в 10 раз меньше
- Кэш ломается от **любого** изменения префикса: даже дата в системном промпте

system + tools + документы

стабильно,
кэшируем

история диалога

новый вопрос

меняется,
в конец

Правило порядка

Стабильное в начало, переменное в конец. Проверять по usage: если cache_read равен нулю на повторных вызовах, что-то в префиксе меняется.

- Ключ только в переменной окружения или `.env`, никогда в коде
- `.env` в `.gitignore` до первого коммита, а не после утечки
- В Kaggle: Add-ons, Secrets. Ноутбуки публикуются вместе с ячейками
- Лимит расходов на ключе: утёкший ключ с лимитом в 5 долларов это неприятность, без лимита катастрофа

Утёк ключ

Отозвать немедленно, выпустить новый, посмотреть расходы за последние часы. Боты сканируют GitHub быстрее, чем вы успеете удалить коммит.

Бесплатные тиры: на чём учиться бесплатно

Провайдер	В минуту	В день или месяц	Подвох
Mistral	2 запроса	до 1 млрд токенов/мес	медленно, зато много
Groq	30 запросов	100–500 тыс токенов/день	только открытые модели
Gemini Flash	10–15 запросов	250–1000 запросов/день	не во всех регионах
OpenRouter :free	20 запросов	50/день, 1000 после \$10	набор моделей плавает
Cerebras	5 запросов	1 млн токенов/день	урезанный контекст

- Лимиты режут без предупреждения: то, что щедро сегодня, завтра усохнет
- Заведите ключи у трёх-четырёх провайдеров и переключайтесь при 429
- Разовые 10 долларов на OpenRouter поднимают бесплатный лимит в 20 раз навсегда

Данные на август 2026, детальный разбор лимитов в материалах к семинару.

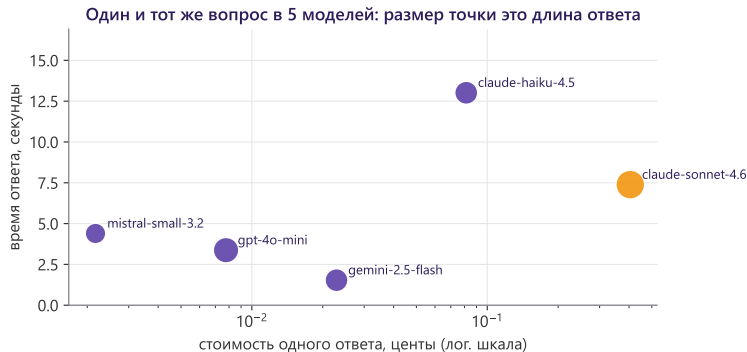
Так не надо

- Задать один вопрос двум моделям и решить «эта умнее»
- Сравнить при разной температуре и разных промптах
- Смотреть только на качество, не на цену и скорость

Так надо

- Фиксированный набор из 20–50 задач с известными ответами
- Одинаковый промпт, температура ноль
- Три числа на модель: доля верных, средняя цена, среднее время

Один вопрос, несколько моделей



- Вопрос про опоздание на стыковочный рейс, одинаковый промпт, температура ноль
- По цене разброс на два порядка, по времени в разы, а ответы на глаз похожи. Различить их можно только на фиксированном наборе задач

- 1 Температура по умолчанию для извлечения данных: ответы гуляют от запуска к запуску
- 2 Парсить JSON из текста регулярками вместо схемы
- 3 Не читать `usage`: бюджет кончается неожиданно
- 4 Класть инструкции в `user` вместо `system`: работает, пока история короткая
- 5 `max_tokens` наугад: ответ обрезан, а код этого не замечает
- 6 Нет повтора при 429: скрипт падает на сотом вызове из тысячи
- 7 Дата или случайный `id` в системном промпте: кэш не работает никогда

Каждую из этих ошибок вы найдёте в чужом коде на семинаре. Некоторые в своём.

Руками

- Ключи у трёх провайдеров, `.env`, Kaggle Secrets
- Первый вызов через `requests` и через SDK
- Structured output с `Pydantic` и повтором при ошибке
- Сравнение трёх моделей на десяти задачах: таблица цена/качество

Домашка выдаётся на семинаре. Она использует всё с этой лекции и готовит почву под цикл агента.

Подготовить до семинара

Python 3.10+, `requests`, `pydantic`
Аккаунт OpenRouter, ключ в `.env`
Аккаунт Kaggle с подтверждённым телефоном: без него нет интернета в ноутбуке

- 1 Модель через API это функция: текст на входе, текст или JSON на выходе. Памяти у неё нет, память это ваш список сообщений
- 2 Платим за токены, выход дороже входа, контекст накапливается. usage читать всегда
- 3 Системный промпт это контракт: роль, задача, формат, границы. Версионировать и тестировать
- 4 Всё, что читает программа, приходит по схеме. Pydantic плюс один повтор с текстом ошибки
- 5 API падает, лимиты режут, ключи утекают. Повторы, таймауты, запасная модель, лимит расходов

Вопросы?

следующая лекция: что такое агент, цикл ReAct и почему четыре шага стоят как восемь